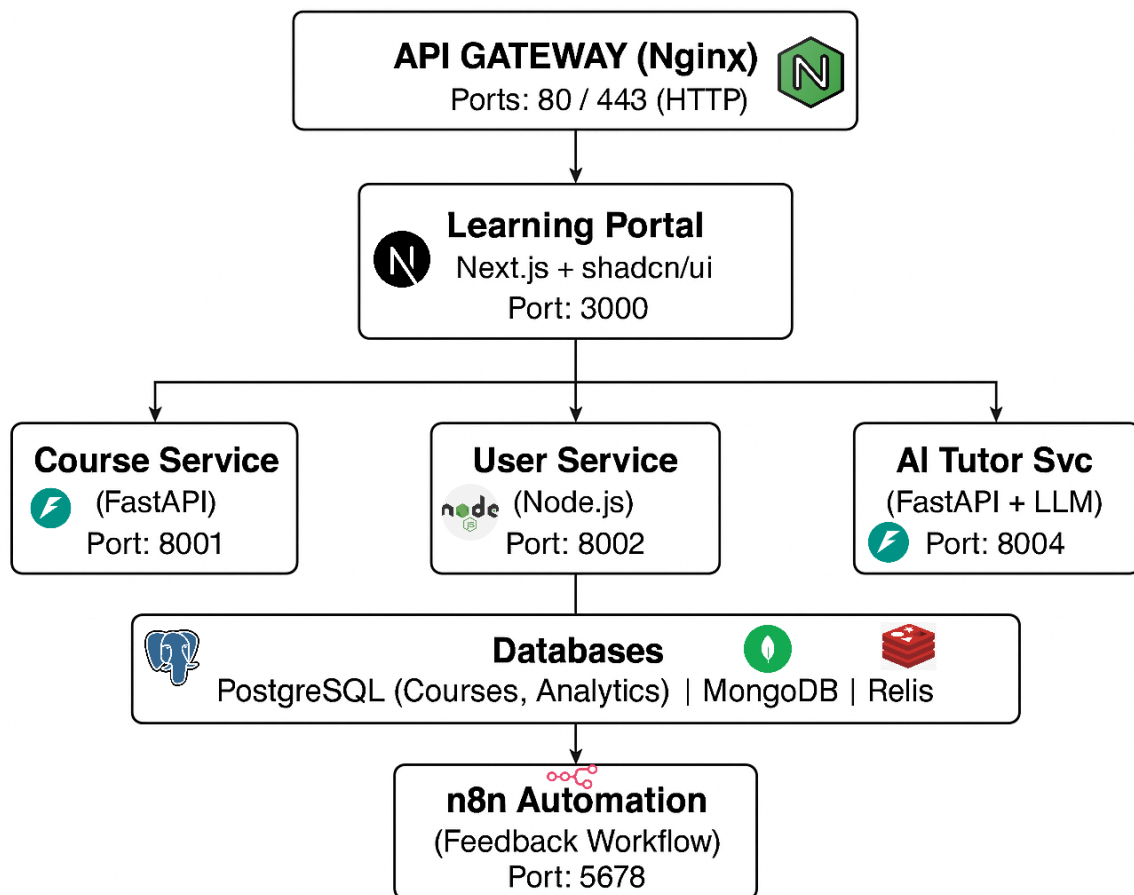


Projet d'intégration de compétences

Créer une plateforme d'apprentissage en ligne (cours gratuits et payants) basée sur une architecture microservices, intégrant une fonctionnalité IA pour assistance pédagogique et un workflow automatisé de collecte de feedback via **n8n**.

Le déploiement se fait avec **Docker Compose**.

Architecture



- **API Gateway (Nginx)** : Reverse proxy, routage, sécurité.
- **Learning Portal (Next.js + shadcn/ui)** : Interface publique pour découvrir, s'inscrire et suivre les cours.
- **Microservices** :
 1. **Course Service (FastAPI)** : Gestion des cours, leçons, inscriptions, recherche.
 2. **User Service (Node.js/Express)** : Authentification JWT, profils, rôles.
 3. **Analytics Service (FastAPI)** : Suivi des vues, inscriptions, tendances.
 4. **AI Tutor Service (FastAPI + LLM local)** : Q&A contextuel, recommandations, génération de quiz.
 5. **n8n Automation** : Workflow de feedback (collecte, analyse IA, stockage, notification).
- **Bases de données** : PostgreSQL (cours, analytics), MongoDB (utilisateurs), Redis (cache), MinIO (stockage médias).

Learning Portal (Next.js)

- Catalogue de cours avec recherche et filtres.
- Pages de cours (syllabus, avis, prix).
- Lecteur de leçons (texte/vidéo) avec suivi de progression.
- Authentification JWT.
- **IA intégrée** : Chat Q&A basé sur le contenu du cours + recommandations personnalisées.

Fonctionnalités Clés

- **IA Tutor** : Répond aux questions des apprenants, génère quiz et recommandations.
- **Workflow n8n** :
 - Déclenchement après feedback → enrichissement → résumé IA → stockage → notification.
- **Analytics** : Tableau de bord (inscriptions, complétions, tendances).

Containerisation

- Services : `learning-frontend`, `course-service`, `user-service`, `analytics-service`, `ai-tutor-service`, `n8n-automation`, `nginx-gateway`.
- Datastores : `postgres`, `mongodb`, `redis`, `minio`.
- Déploiement via **docker-compose.yml**.

Architecture avec C4 et UML

1. Utiliser le C4 Model pour la structuration

- **Niveau 1 (Context Diagram) :**
 - Montrer la plateforme comme un système global.
 - Identifier les acteurs externes (ex. : Apprenant, Instructeur) et leurs interactions avec le système.
- **Niveau 2 (Container Diagram) :**
 - Décomposer le système en conteneurs :
 - Frontend (Learning Portal)
 - Microservices (Course, User, Analytics, AI Tutor, n8n)
 - Bases de données (PostgreSQL, MongoDB, Redis, MinIO)
 - API Gateway (Nginx)
 - Indiquer les technologies (Angular, Next.js, FastAPI, Node.js, etc.).
- **Niveau 3 (Component Diagram) :**
 - Pour chaque microservice, détailler les composants internes (ex. : contrôleurs, services, DAO).
- **Niveau 4 (Code Diagram) (*optionnel*) :**
 - Montrer les classes ou modules clés si nécessaire.

2. Compléter avec UML pour la précision

- **Diagrammes UML recommandés :**
 - **Use Case Diagram :**
 - Cas d'utilisation : inscription, suivi de cours, Q&A IA, feedback.
 - **Class Diagram :**
 - Pour représenter les entités principales (User, Course, Lesson, Feedback).
 - **Sequence Diagram :**
 - Exemple : flux d'inscription → accès au cours → interaction avec AI Tutor → feedback → n8n workflow.
 - **Deployment Diagram :**
 - Montrer les conteneurs Docker, leurs ports, et les liens réseau.

Livrables

- Code source structuré (frontends, services, infra).
- Fichiers `docker-compose.yml` et `README.md` avec instructions.
- Diagramme d'architecture.

Remarques

- Date validation du projet : Février 2026
- Travail individuel